

#线性回归单因子和多因子预测

```
import pandas as pd

import numpy as np

data = pd.read_csv('')

data.head()

%matplotlib inline
import matplotlib import pyplot as plt

fig = plt.figure(figsize=(10,10))

fig1 = plt.subplot(231)

plt.scatter(data.loc[:, 'A'], data.loc[:, 'B'])

plt.title('')

fig2 = plt.subplot(232)

plt.scatter(data.loc[:, 'A'], data.loc[:, 'D'])

plt.title('')

fig3 = plt.subplot(233)

plt.scatter(data.loc[:, 'A'], data.loc[:, 'F'])

plt.title('')

fig4 = plt.subplot(234)

plt.scatter(data.loc[:, 'A'], data.loc[:, 'H'])

plt.title('')

fig5= plt.subplot(235)

plt.scatter(data.loc[:, 'A'], data.loc[:, 'G'])

plt.title('')

x=data.loc[:, 'B']

y=data.loc[:, 'A']

x=np.array(x).reshape(-1,1)

from sklearn.linear_model import LinearRegression
```

```

LR1=LinearRegression()

LR1.fit(X,y)

y_predict_1=LR1.predict(X)

from sklearn.metrics import mean_squared_error,r2_score

mean_squared_erroe_1=mean_squared_error(y,y_predict_1)

r2_score=r2_score(y,y_predict_1)

print(mean_squared_erroe_1,r2_score)


fig6 = plt.figure(figsize=(10,10))

plt.scatter(x, y)

plt.plot(x,y_predict_1,'r')

plt.show()


x_multi=data.drop(['price'],axis=1)

LR_multi = LinearRegression()

LR_multi.fit(x_multi,y)

y_predict_multi=LR_multi.predict(x_multi)

mean_squared_erroe_multi=mean_squared_error(y,y_predict_multi)

r2_score_multi=r2_score(y,y_predict_multi)


fig7 = plt.figure(figsize=(10,10))

plt.scatter(y, y_predict_multi)

plt.show()

##预测是否合理
x_text=[65000,5,5,30000,200]
x_text=np.array(x_text).reshape(-1,1)
y_predict = LR_multi.predict(x_text)

##算ab
a=LR_multi.coef_
b=LR_multi.intercept_

```

#分类任务(逻辑回归)

#初始利用线性查看, 不太好 边界函数: $a_0 + a_1x_1 + a_2x_2 = 0$

```
mask = data.loc[:, 'Pass'] == 1

fig = plt.figure(figsize=(10,10))

passed=plt.scatter(data.loc[:, 'exam1']【mask】, data.loc[:, 'exam2']【mask】)

failed=plt.scatter(data.loc[:, 'exam1']【~mask】, data.loc[:, 'exam2']【~mask】)

plt.legend((passed,failed),('passed','failed'))

plt.show()

x=data.drop(['pass'],axis=1)

y=data.loc[:, 'pass']

x1=data.loc[:, 'exam1']

x2=data.loc[:, 'exam2']

from sklearn.linear_model import LogisticRegression

LR = LogisticRegression()

LR.fit(x,y)

y_predict=LR.predict(x)

from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y,y_predict)

y_test = LR.predict([[70,65]])

print('passed' if y_test==1 else 'failed')
```

#二阶边界函数: $a_0 + a_1x_1 + a_2x_2 + (a_3x_1)^2 + (a_4x_2)^2 + a_5x_1x_2 = 0$

#要先利用x1和x2去造对应的数据, 并计算

```

week3_logistic_task1_exam - x
localhost:8888/notebooks/jupyter/mococourses/week3_logistic_regression/week3_logistic_task1_exam.ipynb
File Edit View Insert Cell Kernel Widgets Help Python 3

In [31]: y2_predict = LR2.predict(X_new)
accuracy2 = accuracy_score(y, y2_predict)
print(accuracy2)

1.0

In [ ]: X1_new = X1.sort_values()
print(X1, X1)

边界函数:  $\theta_0 + \theta_1 X_1 + \theta_2 X_2 = 0$ 

二阶边界函数:  $\theta_0 + \theta_1 X_1 + \theta_2 X_2 + \theta_3 X_1^2 + \theta_4 X_2^2 + \theta_5 X_1 X_2 = 0$ 

 $ax^2 + bx + c = 0: x1 = (-b + (b^2 - 4ac)^{0.5})/2a, x1 = (-b - (b^2 - 4ac)^{0.5})/2a$ 

 $\theta_4 X_2^2 + (\theta_5 X_1 + \theta_2) X_2 + (\theta_0 + \theta_1 X_1 + \theta_3 X_1^2) = 0$ 

In [35]: theta0 = LR2.intercept
theta1, theta2, theta3, theta4, theta5 = LR2.coef_[0][0], LR2.coef_[0][1], LR2.coef_[0][2], LR2.coef_[0][3], LR2.coef_[0][4]
a = theta4
b = theta5 * X1 + theta2
c = theta0 + theta1 * X1 + theta3 * X1 * X1
X2_new_boundary = (-b + np.sqrt(b * b - 4 * a * c)) / (2 * a)
print(X2_new_boundary)

```

```

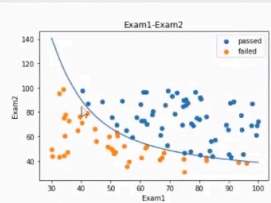
week3_logistic_task1_exam - x
localhost:8888/notebooks/jupyter/mococourses/week3_logistic_regression/week3_logistic_task1_exam.ipynb
File Edit View Insert Cell Kernel Widgets Help Python 3

4 / 39.162968
51 38.844836
97 38.835178
75 38.728953
Name: Exam1, Length: 100, dtype: float64

In [40]: fig5 = plt.figure()
passed = plt.scatter(data.loc[:, 'Exam1'][mask], data.loc[:, 'Exam2'][mask])
failed = plt.scatter(data.loc[:, 'Exam1'][~mask], data.loc[:, 'Exam2'][~mask])
plt.plot(X1_new, X2_new_boundary)
plt.title('Exam1-Exam2')
plt.xlabel('Exam1')
plt.ylabel('Exam2')
plt.legend((passed, failed), ('passed', 'failed'))
plt.show()

plt.plot(X1_new, X2_new_boundary)
plt.show()

```

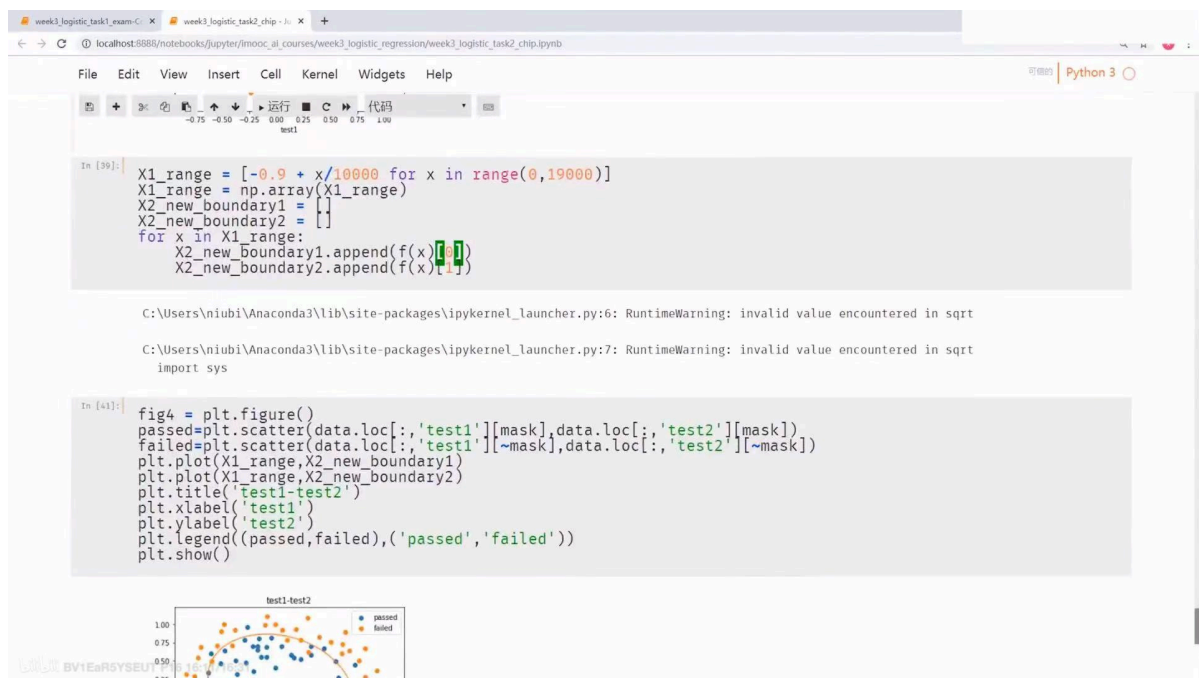


##逻辑回归预测芯片是否通过(以函数的方法)

```

def f(x):
    a=theta4
    b=theta5*x+theta2
    c=theta0+theta1*x+theta3*x*x
    x2_new_boundary1=(-b+np.sqrt(b*b-4*a*c))/(2*a)
    x2_new_boundary2=(-b-np.sqrt(b*b-4*a*c))/(2*a)
    return x2_new_boundary1,x2_new_boundary2
##f(x)[0]为x2_new_boundary1, f(x)[1]为x2_new_boundary2
x2_new_boundary1=[]
x2_new_boundary2=[]
for x in x1_new:
    x2_new_boundary1.append(f(x)[0])
    x2_new_boundary2.append(f(x)[1])
print(x2_new_boundary1,x2_new_boundary2)

```



无监督学习

#kmeans聚类（数据与中心点距离划分，手动指定类别数量）

K均值聚类

K-均值算法：以空间中k个点为中心进行聚类，对最靠近他们的对象归类，是聚类算法中最为基础但也最为重要的算法。

公式：

数据点与各簇中心点距离： $dist(x_i, u_j^t)$

根据距离归类： $x_i \in u_{nearest}^t$

中心更新： $u_j^{t+1} = \frac{1}{k} \sum_{x_i \in S_j} (x_i)$

S_j : t时刻第j个区域簇；k: 包含在 S_j 范围内点的个数； x_i : 包含在 S_j 范围内的点； u_j^t 为t状态下第j区域中心

慕课网

#meanshift均值漂移聚类（中心点一定区域检索数据点，自动发现类别数量）

#DBSCAN算法（基于密度的空间聚类算法，过滤掉噪音数据）

##2D数据类别划分

```

import pandas as pd
import numpy as np
data = pd.read_csv('data.csv') #有三列，两列数据，一列label

#定义 x and y

```

```

x = data.drop(['label'],axis=1)
y = data.loc[:, 'label']
print(x.shape,y.shape)

#set the model
from sklearn.cluster import KMeans
KM = KMeans(n_clusters=3,random_state=0)
KM.fit(x)

centers=KM.cluster_centers_

#图形查看
%matplotlib inline
from matplotlib import pyplot as plt
fig3 = plt.figure()
label0 = plt.scatter(x.loc[:, 'v1'][y=0],x.loc[:, 'v2'][y=0])
label1 = plt.scatter(x.loc[:, 'v1'][y=1],x.loc[:, 'v2'][y=1])
label2 = plt.scatter(x.loc[:, 'v1'][y=2],x.loc[:, 'v2'][y=2])
plt.title("label data")
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend((label0,label1,label2),('label0','label1','label2'))
plt.scatter(centers[:,0],centers[:,1])
plt.show()

#test_data验证并预测准确性,发现准确性不高,是因为cluster不对应
y_predict = KM.predict(x)
print(pd.value_counts(y_predict),pd.value_counts(y))
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y,y_predict)
print(accuracy)

#图形查看
fig4 = plt.subplot(121)
label0 = plt.scatter(x.loc[:, 'v1'][y_predict=0],x.loc[:, 'v2'][y_predict=0])
label1 = plt.scatter(x.loc[:, 'v1'][y_predict=1],x.loc[:, 'v2'][y_predict=1])
label2 = plt.scatter(x.loc[:, 'v1'][y_predict=2],x.loc[:, 'v2'][y_predict=2])
plt.title("predict data")
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend((label0,label1,label2),('label0','label1','label2'))
plt.scatter(centers[:,0],centers[:,1])

fig5 = plt.subplot(122)
label0 = plt.scatter(x.loc[:, 'v1'][y=0],x.loc[:, 'v2'][y=0])
label1 = plt.scatter(x.loc[:, 'v1'][y=1],x.loc[:, 'v2'][y=1])
label2 = plt.scatter(x.loc[:, 'v1'][y=2],x.loc[:, 'v2'][y=2])
plt.title("label data")
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend((label0,label1,label2),('label0','label1','label2'))
plt.scatter(centers[:,0],centers[:,1])
plt.show()

#矫正correct the results
y_correct = []

```

```

for i in y_predict:
    if i==0:
        y_correct.append(1)
    elif i==1:
        y_correct.append(2)
    else:
        y_correct.append(0)
print(pd.value_counts(y_correct),pd.value_counts(y))
print(accuracy_score(y,y_correct))
y_correct = np.array(y_correct)
print(type(y_correct))

#picture
fig6 = plt.subplot(121)
label0 = plt.scatter(x.loc[:, 'v1'][y_correct=0],x.loc[:, 'v2'][y_correct=0])
label1 = plt.scatter(x.loc[:, 'v1'][y_correct=1],x.loc[:, 'v2'][y_correct=1])
label2 = plt.scatter(x.loc[:, 'v1'][y_correct=2],x.loc[:, 'v2'][y_correct=2])
plt.title("predict data")
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend((label0,label1,label2),('label0','label1','label2'))
plt.scatter(centers[:,0],centers[:,1])

fig7 = plt.subplot(122)
label0 = plt.scatter(x.loc[:, 'v1'][y=0],x.loc[:, 'v2'][y=0])
label1 = plt.scatter(x.loc[:, 'v1'][y=1],x.loc[:, 'v2'][y=1])
label2 = plt.scatter(x.loc[:, 'v1'][y=2],x.loc[:, 'v2'][y=2])
plt.title("label data")
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend((label0,label1,label2),('label0','label1','label2'))
plt.scatter(centers[:,0],centers[:,1])
plt.show()

#####
#establish a KNNmodel
from sklearn.neighbors import KNeighborsClassifier
KNN = KNeighborsClassifier(n_neighbors=3)
KNN.fit(x,y)

#predict based on the test data v1=80 v2=60
y_predict_knn_test = KNN.predict([80,60])
y_predict_knn = KNN.predict(x)
print(y_predict_knn_test)
print('knn accuracy:',accuracy_score(y,y_predict_knn))
print(pd.value_counts(y_predict_knn),pd.value_counts(y))

#####
#try meanshift model
from sklearn.cluster import MeanShift,estimate_bandwidth
bw = estimate_bandwidth(x,n_sample=500) #圆的半径
ms = MeanShift(bandwidth=bw)
ms.fit(x)
y_predict_ms = ms.predict(x)
print(pd.value_counts(y_predict_ms),pd.value_counts(y))
后续需要矫正

```

.....

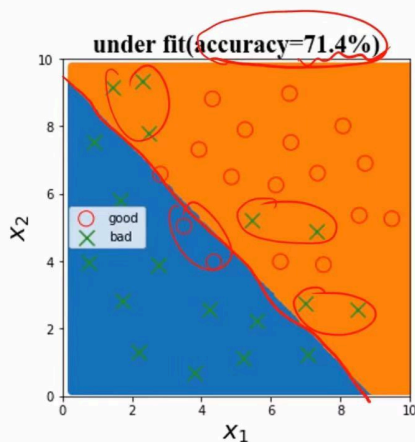
##决策树；异常检测；主成分分析；数据降维

```
#有四个数据加上一列分类的数据降维处理
from sklearn.preprocessing import StandardScaler
x_norm = StandardScaler().fit_transform(x)
from sklearn.decomposition import PCA
pca = PCA(n_components=4)
x_reduced = pca.fit_transform(x_norm)
# 查看各成分的方差比例
var_ratio = pca.explained_variance_ratio_
plt.bar([1,2,3,4],var_ratio)
plt.title('')
plt.xticks([1,2,3,4],['PC1','PC2','PC3','PC4'])
plt.ylabel('var_ratio')
plt.show()
##降维
pca = PCA(n_components=2)
x_reduced = pca.fit_transform(x_norm)
fig3 = plt.figure(figsize=(10,10))
setosa = plt.scatter(x_pca[:,0][y==0],x_pca[:,1][y==0])
versicolor = plt.scatter(x_pca[:,0][y==1],x_pca[:,1][y==1])
virginica = plt.scatter(x_pca[:,0][y==2],x_pca[:,1][y==2])
plt.show()

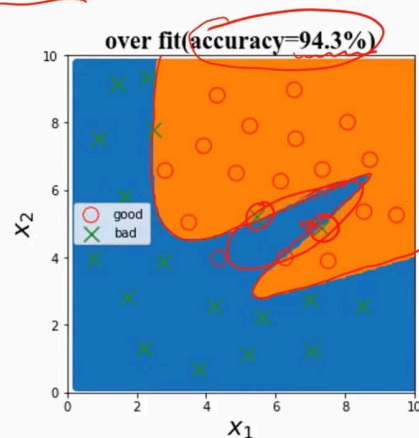
#calculate the gaussian distribution p(x)
x1_mean=x1.mean()
x1_std=x1.std()
from scipy.stats import norm
x1_range = np.linspace(0,20,300)
x1_normal = norm.pdf(x1_range,x1_mean,x1_std)
```

#欠拟合和过拟合

欠拟合与过拟合



决策边界: $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$



决策边界:

$$f((\theta_0 \dots \theta_{21}), (x_1 \dots x_1^5), (x_2 \dots x_2^5)) = n$$

欠拟合与过拟合

模型不合适，导致其无法对数据实现有效预测。

模型对数据的预测情况

	<u>训练数据</u>	<u>预测数据</u>
<u>欠拟合</u>	<u>不准确</u>	<u>不准确</u>
过拟合	准确	不准确
<u>好模型</u>	<u>准确</u>	<u>准确</u>